

Kazakh-British Technical University

Faculty of Information Technology

LABORATORY WORK 1

ERD DIAGRAM - INTERNATIONAL AIRPORT

EMILIYA RODNOVA

Almaty, 2026

1. Introduction

This laboratory work designs and implements a simplified international airport database in PostgreSQL. The solution identifies ten entities, defines their attributes and keys, and connects them through fourteen foreign-key constraints. The editable ER diagram and SQL files use the same table and column names.

2. Identified entities

Entity	Purpose
airport	Stores departure and arrival airports.
airline	Stores airline companies.
flight	Stores one dated flight occurrence, not a recurring route.
passenger	Stores one profile per passport in its issuing country.
booking	Connects one passenger to one flight and stores booking details.
booking_changes	Automatically records every business-field update to a booking.
boarding_pass	Stores at most one current boarding pass per booking.
baggage	Stores each individual registered baggage item.
baggage_check	Stores checks of a specific baggage item and its owner.
security_check	Stores passenger security screening attempts.

A Flight row represents one dated flight. A Booking row connects one passenger with one flight. The model includes baggage, passenger checks and an automatic booking-change history. Airports are linked to flights twice: once for departure and once for arrival.

3. Attributes, keys and data types

PK = primary key; FK = foreign key; NN = NOT NULL; UQ = single-column uniqueness; NULL = optional. All PKs use BIGINT GENERATED ALWAYS AS IDENTITY. TIMESTAMPTZ stores an instant with time-zone awareness. Composite constraints are listed below each table.

airport

Attribute	Type	Rules	Description
airport_id	BIGINT	PK	Automatically generated unique identifier.
airport_name	VARCHAR(150)	NN	Airport name.
country	VARCHAR(100)	NN	Country name.
state	VARCHAR(100)	NULL	State or region, if applicable.
city	VARCHAR(100)	NN	City served by the airport.
created_at	TIMESTAMPTZ	NN	Record creation time; default current timestamp.
updated_at	TIMESTAMPTZ	NN	Last modification time; automatically updated.

airline

Attribute	Type	Rules	Description
airline_id	BIGINT	PK	Automatically generated unique identifier.
airline_code	VARCHAR(10)	NN, UQ	Unique airline code within this system.
name	VARCHAR(100)	NN	Airline name.
country	VARCHAR(100)	NN	Country of the airline.
created_at	TIMESTAMPTZ	NN	Record creation time; default current timestamp.
updated_at	TIMESTAMPTZ	NN	Last modification time; automatically updated.

flight

Attribute	Type	Rules	Description
flight_id	BIGINT	PK	Automatically generated unique identifier.
departing_gate	VARCHAR(10)	NULL	Departure gate; optional until assigned.
arriving_gate	VARCHAR(10)	NULL	Arrival gate; optional until assigned.
airline_id	BIGINT	FK, NN	References airline.airline_id.
departure_airport_id	BIGINT	FK, NN	References airport.airport_id.
arrival_airport_id	BIGINT	FK, NN	References airport.airport_id.
scheduled_departure_time	TIMESTAMPTZ	NN	Planned departure instant.
scheduled_arrival_time	TIMESTAMPTZ	NN	Planned arrival instant.
actual_departure_time	TIMESTAMPTZ	NULL	Actual departure; unknown before departure.
actual_arrival_time	TIMESTAMPTZ	NULL	Actual arrival; requires actual departure.
created_at	TIMESTAMPTZ	NN	Record creation time; default current timestamp.
updated_at	TIMESTAMPTZ	NN	Last modification time; automatically updated.

passenger

Attribute	Type	Rules	Description
passenger_id	BIGINT	PK	Automatically generated unique identifier.
first_name	VARCHAR(100)	NN	Given name.
last_name	VARCHAR(100)	NN	Family name.
gender	VARCHAR(20)	NULL	Gender, if provided.
date_of_birth	DATE	NN	Birth date; cannot be after the profile creation date.
country_of_citizenship	VARCHAR(100)	NN	Citizenship country.
country_of_residence	VARCHAR(100)	NN	Residence country.
passport_issuing_country	VARCHAR(100)	NN	Issuing country; added to scope passport uniqueness.
passport_number	VARCHAR(30)	NN	Passport number as text, preserving leading zeros.
created_at	TIMESTAMPTZ	NN	Record creation time; default current timestamp.
updated_at	TIMESTAMPTZ	NN	Last modification time; automatically updated.

Table constraints: UNIQUE (passport_issuing_country, passport_number).

booking

Attribute	Type	Rules	Description
booking_id	BIGINT	PK	Automatically generated unique identifier.
flight_id	BIGINT	FK, NN	References flight.flight_id.
passenger_id	BIGINT	FK, NN	References passenger.passenger_id.
status	VARCHAR(30)	NN	pending, confirmed or cancelled; default pending.
booking_platform	VARCHAR(50)	NN	Booking channel, for example website.
ticket_price	NUMERIC(10,2)	NN	Non-negative finite ticket price in USD.
created_at	TIMESTAMPTZ	NN	Record creation time; default current timestamp.
updated_at	TIMESTAMPTZ	NN	Last modification time; automatically updated.

Table constraints: UNIQUE (booking_id, passenger_id); UNIQUE (booking_id, flight_id); UNIQUE active (flight_id, passenger_id) WHERE status <> cancelled.

booking_changes

Attribute	Type	Rules	Description
change_id	BIGINT	PK	Automatically generated unique identifier.
booking_id	BIGINT	FK, NN	References booking.booking_id.
old_flight_id	BIGINT	FK, NN	References flight.flight_id.
new_flight_id	BIGINT	FK, NN	References flight.flight_id.
change_reason	VARCHAR(255)	NN	Automatic description of the changed booking.
old_values	JSONB	NN	Previous business-field snapshot, excluding audit timestamps.
new_values	JSONB	NN	New business-field snapshot, excluding audit timestamps.
changed_at	TIMESTAMPTZ	NN	Automatically assigned change timestamp.

boarding_pass

Attribute	Type	Rules	Description
boarding_pass_id	BIGINT	PK	Automatically generated unique identifier.
booking_id	BIGINT	FK, NN, UQ	Part of composite FK to booking; one pass per booking.
flight_id	BIGINT	FK, NN	Together with booking_id references booking; enables seat uniqueness.
seat	VARCHAR(10)	NN	Uppercase seat such as 12A; unique within a flight.
boarding_time	TIMESTAMPTZ	NN	Planned boarding instant.
created_at	TIMESTAMPTZ	NN	Record creation time; default current timestamp.
updated_at	TIMESTAMPTZ	NN	Last modification time; automatically updated.

Table constraints: FK (booking_id, flight_id) -> booking (booking_id, flight_id); UNIQUE (flight_id, seat).

baggage

Attribute	Type	Rules	Description
baggage_id	BIGINT	PK	Automatically generated unique identifier.
booking_id	BIGINT	FK, NN	References booking.booking_id.
weight_kg	NUMERIC(6,2)	NN	Positive finite weight in kilograms.
created_at	TIMESTAMPTZ	NN	Record creation time; default current timestamp.
updated_at	TIMESTAMPTZ	NN	Last modification time; automatically updated.

Table constraints: UNIQUE (baggage_id, booking_id).

baggage_check

Attribute	Type	Rules	Description
baggage_checking_id	BIGINT	PK	Automatically generated unique identifier.
baggage_id	BIGINT	FK, NN	Composite FK to baggage identifies the checked item.
booking_id	BIGINT	FK, NN	Shared by the composite FKs to baggage and booking.
passenger_id	BIGINT	FK, NN	References passenger and belongs to the referenced booking.
check_results	VARCHAR(30)	NN	pending, passed or failed; default pending.
created_at	TIMESTAMPTZ	NN	Record creation time; default current timestamp.
updated_at	TIMESTAMPTZ	NN	Last modification time; automatically updated.

Table constraints: FK (baggage_id, booking_id) -> baggage (baggage_id, booking_id); FK (booking_id, passenger_id) -> booking (booking_id, passenger_id).

security_check

Attribute	Type	Rules	Description
security_check_id	BIGINT	PK	Automatically generated unique identifier.
passenger_id	BIGINT	FK, NN	References passenger.passenger_id.
check_results	VARCHAR(30)	NN	pending, passed or failed; default pending.
created_at	TIMESTAMPTZ	NN	Record creation time; default current timestamp.
updated_at	TIMESTAMPTZ	NN	Last modification time; automatically updated.

4. Relationships and cardinalities

In every relationship, each child row must have exactly one parent. A parent may exist without any children. All foreign-key columns are NOT NULL. R06 has at most one child because `boarding_pass.booking_id` is UNIQUE.

ID	Parent -> child	Children per parent	Foreign-key columns in child
R01	airport -> flight	0..N	<code>departure_airport_id</code>
R02	airport -> flight	0..N	<code>arrival_airport_id</code>
R03	airline -> flight	0..N	<code>airline_id</code>
R04	flight -> booking	0..N	<code>flight_id</code>
R05	passenger -> booking	0..N	<code>passenger_id</code>
R06	booking -> boarding_pass	0..1	<code>booking_id</code> , <code>flight_id</code>
R07	booking -> baggage	0..N	<code>booking_id</code>
R08	booking -> booking_changes	0..N	<code>booking_id</code>
R09	flight -> booking_changes	0..N	<code>old_flight_id</code>
R10	flight -> booking_changes	0..N	<code>new_flight_id</code>
R11	baggage -> baggage_check	0..N	<code>baggage_id</code> , <code>booking_id</code>
R12	booking -> baggage_check	0..N	<code>booking_id</code> , <code>passenger_id</code>
R13	passenger -> baggage_check	0..N	<code>passenger_id</code>
R14	passenger -> security_check	0..N	<code>passenger_id</code>

R06, R11 and R12 use composite foreign keys. R06 ensures that a boarding pass uses its booking's flight. R11 ensures that baggage belongs to the same booking as its check. R12 ensures that the passenger owns that booking. The exact referenced columns are defined in the SQL.

Passenger and Flight have a conceptual many-to-many relationship resolved through Booking. A partial unique index permits at most one non-cancelled booking per passenger and flight. Referenced records cannot be deleted or have their referenced keys changed while dependants remain.

5. ER diagram and notation

The following landscape sheet contains the complete ER diagram. The native Airport_ERD.drawio file is editable: tables, field labels and connections are individual diagram objects. The landscape sheet uses a larger page size so the complete structure can be zoomed and read.

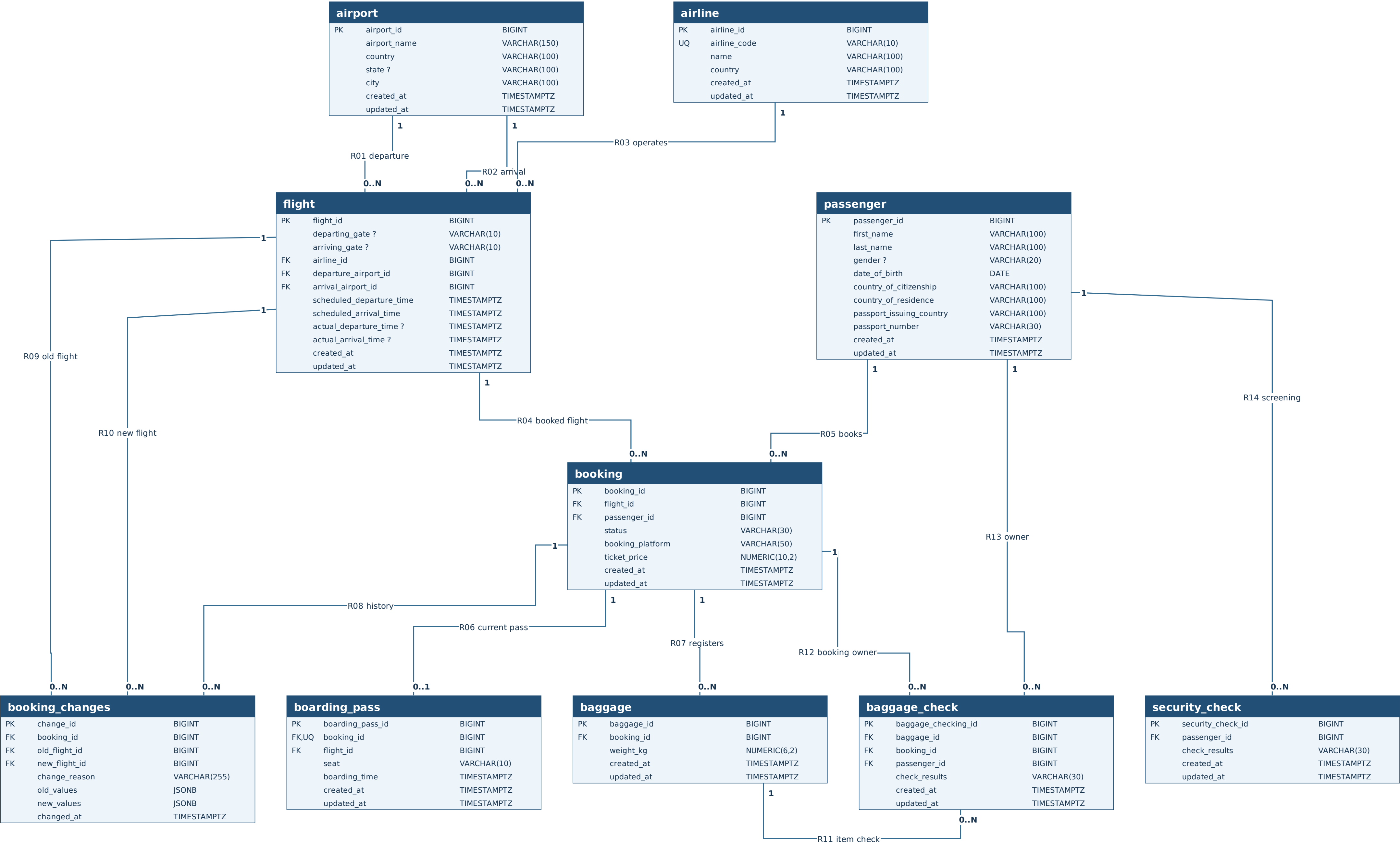
Notation	Meaning
PK	Primary key: uniquely identifies a record and cannot be NULL.
FK	Foreign key: references an existing parent record.
UQ	A single-column UNIQUE constraint. Composite uniqueness is described in Section 3.
?	Nullable field on the diagram. Other fields are required.
1	Exactly one parent record.
0..N	Zero, one or many related child records.
0..1	Zero or one related child record.
R01-R14	Relationship identifiers matching the matrix in Section 4.

Consistency: All 76 modelled fields were compared with the created PostgreSQL catalog. Table names use lowercase snake_case to avoid quoted, case-sensitive SQL identifiers.

The complete ERD is on the next page.

INTERNATIONAL AIRPORT | PostgreSQL ER Diagram

10 tables / 14 foreign-key relationships / Emiliya Rodnova / R labels correspond to the report



PK = primary key | FK = foreign key | UQ = unique | ? = nullable | Unmarked fields are required
1 = exactly one; 0..N = zero or many; 0..1 = zero or one. Composite foreign keys are detailed in the report.

6. Design assumptions and integrity rules

Passport uniqueness. A passport number alone is not assumed globally unique. `passport_issuing_country` is added, and the pair (`passport_issuing_country`, `passport_number`) is unique. Countries are stored as consistently spelled names.

Booking changes. The source requires a separate history table but does not fully specify its fields. `old_flight_id` and `new_flight_id` preserve the reference example; JSONB snapshots also capture changes to status, price and platform. A trigger adds a history row only when business fields change. UPDATE and DELETE on history are rejected by a trigger.

Boarding passes and seats. One booking has zero or one current pass. `flight_id` is added to `boarding_pass`, constrained to its booking through a composite foreign key. `UNIQUE(flight_id, seat)` prevents duplicate seat allocation on a flight. Only confirmed bookings can receive a pass.

Baggage consistency. `baggage_id` is added to `baggage_check` to identify the checked item. Composite foreign keys enforce matching baggage, booking and passenger. Repeated check attempts are allowed. A check requires a confirmed booking; zero checks is possible before airport processing.

Rebooking and cancellation. The passenger of an existing booking cannot be changed. A flight change deletes the old boarding pass and writes history automatically. Flight changes after baggage screening are rejected; a new booking is required. Moving a booking away from confirmed status removes its pass.

Times, prices and optional data. All instants use `TIMESTAMPTZ`; example timestamps include `+00 UTC`. The single system currency is USD. Price must be non-negative and baggage weight positive; NaN is rejected. Planned arrival follows departure. Actual arrival requires and follows actual departure. Gates, actual times, state and gender may be NULL.

Scope boundaries. This is a complete implementation of the simplified lab model. Accounts, booking legs, aircraft, loyalty programs, seat maps, payments, authorization roles and multi-currency processing are outside scope. The schema owner retains normal PostgreSQL administrative privileges; the history trigger is not an administrative tamper-proof audit system.

7. SQL implementation and validation

Airport_All_In_One.sql creates the schema, loads demonstration data and executes six example queries. The archive also contains the same work split into schema, demo data, queries and validation scripts. No destructive DROP command is included.

Component	Implemented behaviour
Tables and keys	10 tables; 76 columns; 14 foreign keys; identity primary keys.
Constraints	NOT NULL, UNIQUE, composite foreign keys, CHECK constraints and one partial unique index.
Triggers	Automatic updated_at, booking history, history protection, pass validation and cleanup, baggage-check validation.
Demo data	3 airports, 2 airlines, 3 flights, 3 passengers, 3 bookings, 2 passes, 3 baggage items, 3 baggage checks, 2 security checks and 2 generated history rows.
Example queries	Booking itinerary, baggage totals, history, seats, latest baggage-check result and active bookings per flight.

Executed validation: 31 of 31 checks passed on PostgreSQL 18.3 via PGLite 0.5.8. Schema creation, seed data and all example queries executed successfully. This verifies SQL behaviour on a PostgreSQL engine; a separate native PostgreSQL server and multi-session load tests were not run.

The tests cover invalid values, missing parents, duplicate seats and active bookings, mismatched baggage owners, pass/flight consistency, history protection, rebooking, cancellation and no-op updates. 04_validation_tests.sql rolls back its data changes. Identity sequences can still advance during rolled-back tests.

8. Conclusion

The database implements the required airport workflow with consistent relationships and automatic validation. The editable ERD, data dictionary and executable SQL describe the same model. Additional fields and assumptions are documented explicitly rather than left implicit.

References

Assignment: supplied Laboratory work 1 screenshots.
Formatting reference: supplied DB_Lab1.pdf.
PostgreSQL constraints: [official documentation](#).